grid really is an actual object, unlike in the MNIST example. This means we can see exactly which objects the agent is paying attention to. We might hope it learns to pay most attention to the key, door, and goal square, and that the key is related to the door. If this turns out to be the case, it suggests the agent is learning not too differently than how a human would learn how to relate the objects on the grid.

Overall we will repurpose the relational module we created earlier for the MNIST example as a relational DQN, so we really only need to change the output to a normal activation function rather than the `log_softmax` we used for classification. But first, let's get back to implementing multi-head attention. As operating on higher-order tensors gets more complicated, we will get help from a package called Einops that extends the capabilities of PyTorch's built-in Einsum function. You can install it using `pip`:

```
>>> pip install einops
>>> from einops import rearrange
```

There are only two important functions in this package (`rearrange` and `reduce`), and we will only use one, the `rearrange` function. `rearrange` basically lets us reshape the dimensions of a higher-order tensor more easily and readably than the built-in PyTorch functions, and it has a syntax similar to Einsum. For example, we can reorder the dimensions of a tensor like this:

```
>>> x = torch.randn(5,7,7,3)
>>> rearrange(x, "batch h w c -> batch c h w").shape
torch.Size([5, 3, 7, 7])
```

Or if we had collapsed the spatial dimensions *h* and *w* into a single dimension *N* for nodes, we can undo this:

```
>>> x = torch.randn(5,49,3)
>>> rearrange(x, "batch (h w) c -> batch h w c", h=7).shape
torch.Size([5, 7, 7, 3])
```

In this case, we tell it that the input has three dimensions but the second dimension is secretly two dimensions (*h*, *w*) that were collapsed, and we